Lecture 2

Linear Regression

Outline

- **Linear Regression**
  - Simple Example
  - Model
- **Learning**
  - Closed Form
  - Gradient Descent
  - SGD
- **Advanced Topics**
  - Probabilistic Interpretation of LMS
  - L2 Regularization
  - L1 Regularization

Linear Regression

- Given a data set $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$ where $\boldsymbol{x}_i = (x_{i1}; x_{i2}; \ldots; x_{id}) \quad y_i \in \mathbb{R}$
- The aim of linear regression
  - Learn a **linear model** that can accurately predict the real-valued output labels
- For discrete variables (categorical features)
  - An ordinal relationship exists between values
    - Convert the variables into numerical variables (map each rank to an integer)
  - No ordinal relationship exists
    - Convert the discrete variable with *k* possible values into a *k*-dimensional vector (one-hot encoding)

Linear Regression

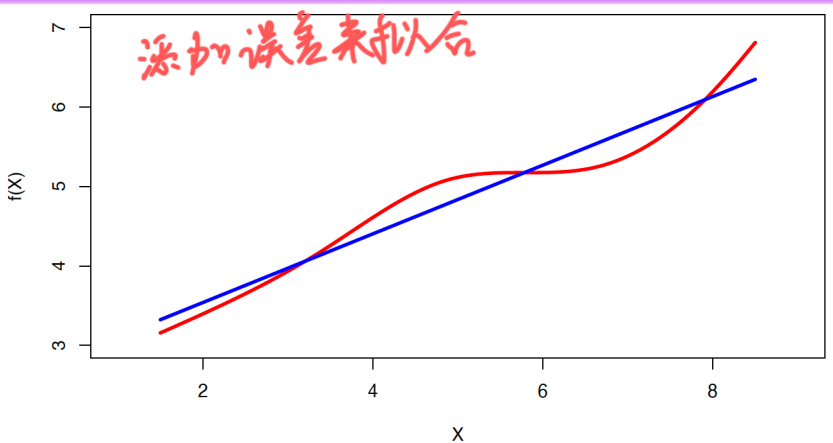
- Linear regression aims to learn the function:

$$f(x) = wx + b, \text{ such that } f(x_i) \simeq y_i, i = 1, \ldots, m$$

- True regression functions are never linear!

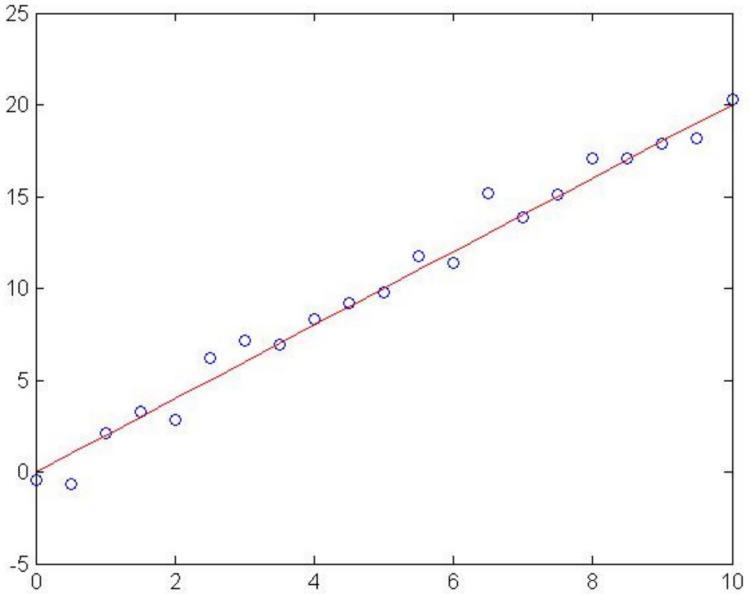
$$y = wx + b + \epsilon$$

where ϵ is an error term of measurement error or other noise



Linear Regression example

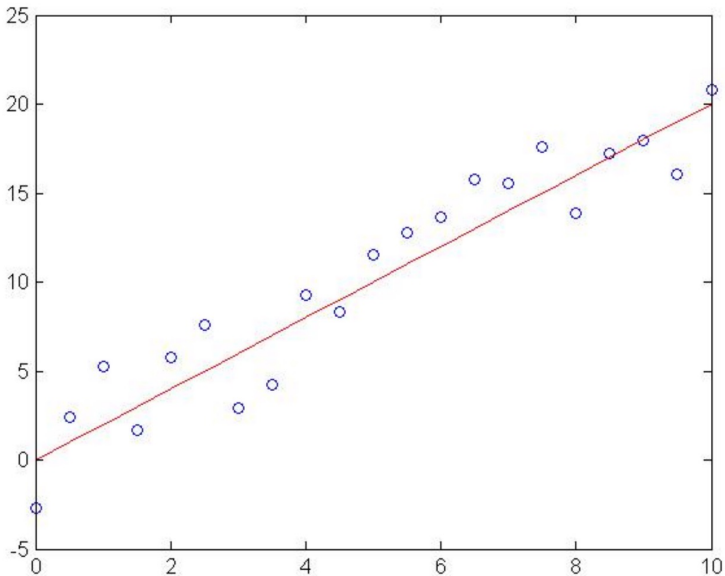
- Generated: w=2
- Recovered: w=2.03
- Noise: std=1



Slide courtesy of William Cohen

Linear Regression example

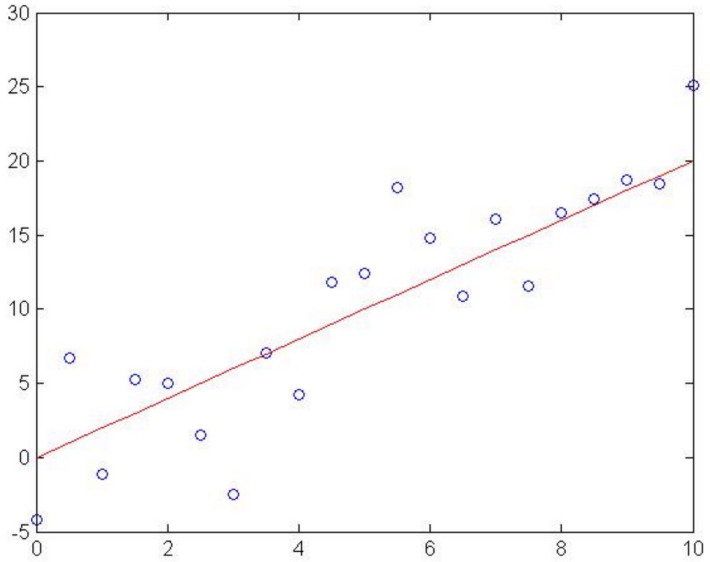
- Generated: w=2
- Recovered: w=2.05
- Noise: std=2



Slide courtesy of William Cohen

Linear Regression example

- Generated: w=2
- Recovered: w=2.08
- Noise: std=4



Slide courtesy of William Cohen

Linear Regression

Data: Inputs are continuous vectors of length d. Outputs are continuous scalars.

$$\mathcal{D} = \left\{ \boldsymbol{x}^{(i)}, y^{(i)} \right\}_{i=1}^{m} \text{ where } \boldsymbol{x} \in \mathbb{R}^d \text{ and } y \in \mathbb{R}$$

Prediction: Output is a linear function of the inputs.

$$\hat{y} = f_\theta(\boldsymbol{x}) = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$$

$$\hat{y} = f_\theta(\boldsymbol{x}) = \boldsymbol{\theta}^T \boldsymbol{x}$$

(We assume x_1 is 1)

Learning: finds the parameters that minimize some objective function. 模型 目标：

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Least Squares

- Our goal is to learn a linear function:

$$f(x) = wx + b, \text{ such that } f(x_i) \simeq y_i, i = 1, \ldots, m$$

- We minimize the sum of the squares:

$J(\boldsymbol{\theta})$ ⇨ $(w^*, b^*) = \arg\min_{(w,b)} \sum_{i=1}^{m} (f(x_i) - y_i)^2$

$\boldsymbol{\theta} = \begin{bmatrix} \boldsymbol{w} \\ b \end{bmatrix}$

$= \arg\min_{(w,b)} \sum_{i=1}^{m} (y_i - wx_i - b)^2$

Reduces distance between true measurements and predicted hyperplane (line in 1D)

Least Squares

Learning: Three approaches to solving $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- **Approach 1:** Closed Form
  (set derivatives equal to zero and solve for parameters)
- **Approach 2:** Gradient Descent
  (take larger – more certain – steps opposite the gradient)
- **Approach 3:** Stochastic Gradient Descent (SGD)
  (take many small steps opposite the gradient)

Linear Regression – loss function

最小均方误差

- Minimize mean-squared error (MSE):

  Loss function: How much $\hat{y}$ differs from the true y

$$E_{(w,b)} = \sum_{i=1}^{m} (y_i - wx_i - b)^2$$

- Calculate the derivatives of $E_{(w,b)}$ with respect to w and b : 求导获得最优解

$$\frac{\partial E_{(w,b)}}{\partial w} = 2\left(w\sum_{i=1}^{m} x_i^2 - \sum_{i=1}^{m}(y_i - b)x_i\right)$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2\left(mb - \sum_{i=1}^{m}(y_i - wx_i)\right)$$

Linear Regression - Least Square Method

- We have the closed-form solutions

闭式解：

$$w = \frac{\sum_{i=1}^{m} y_i(x_i - \bar{x})}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2}$$

$$b = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i)$$

where $\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i$

Multivariate Linear Regression

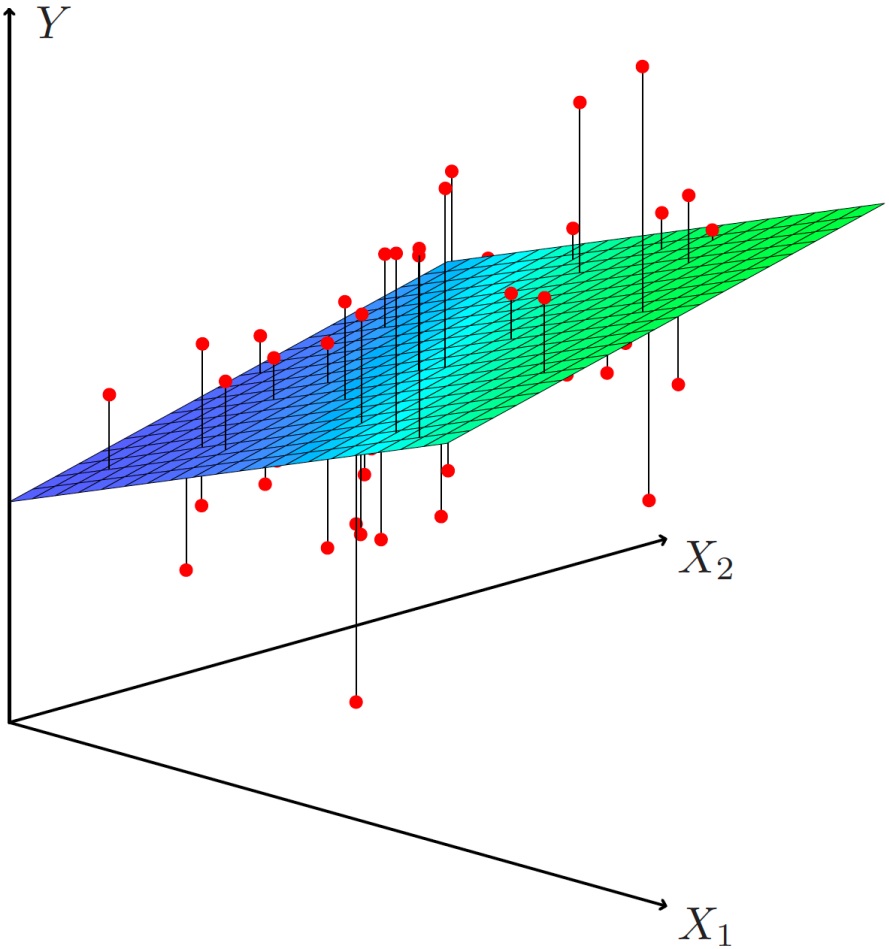
- Given a data set

$$D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$$

$$\boldsymbol{x}_i = (x_{i1}; x_{i2}; \ldots; x_{id}) \quad y_i \in \mathbb{R}$$

- The objective of multivariate linear regression

Find $f(\boldsymbol{x}_i) = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}_i + b$ such that $f(\boldsymbol{x}_i) \simeq y_i$


Y
X2
X1

Multivariate Linear Regression

- Rewrite $\boldsymbol{w}$ and b as $\hat{\boldsymbol{w}} = (\boldsymbol{w}; b)$, the data set is represented as

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^{\mathrm{T}} & 1 \\ \boldsymbol{x}_2^{\mathrm{T}} & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_m^{\mathrm{T}} & 1 \end{pmatrix}$$

$$\boldsymbol{y} = (y_1; y_2; \ldots; y_m)$$

Multivariate Linear Regression - Least Square Method

- Least square method

$$\hat{\boldsymbol{w}}^* = \arg\min_{\hat{\boldsymbol{w}}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})$$

Let $E_{\hat{\boldsymbol{w}}} = (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})$ and find the derivative $\hat{\boldsymbol{w}}$ with respect to

$$\frac{\partial E_{\hat{\boldsymbol{w}}}}{\partial \hat{\boldsymbol{w}}} = 2\mathbf{X}^{\mathrm{T}} (\mathbf{X}\hat{\boldsymbol{w}} - \boldsymbol{y})$$

The closed-form solution of $\hat{\boldsymbol{w}}$ can be obtained by making the equation equal to 0.

Multivariate Linear Regression - Least Square Method

- If $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is a full-rank matrix or a positive definite matrix, then

$$\hat{\boldsymbol{w}}^{*} = \left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}\mathbf{X}^{\mathrm{T}}\boldsymbol{y}$$

where $\left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}$ is the inverse of $\mathbf{X}^{\mathrm{T}}\mathbf{X}$, the learned multivariate linear regression model is

$$f\left(\hat{\boldsymbol{x}}_i\right) = \hat{\boldsymbol{x}}_i^{\mathrm{T}}\left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}\mathbf{X}^{\mathrm{T}}\boldsymbol{y}$$

- $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is often not full-rank
  - gradient descent (which is more broadly applicable)
  - pseudo-inverse

过程：

$$w^* = \arg\min_w (y - X\hat{w})^T (y - X\hat{w}) = \arg\min_w E_{\hat{w}}$$

$$\frac{\partial E_{\hat{w}}}{\partial \hat{w}} = \frac{(y^T - \hat{w}^T X^T)(y - Xw)}{\partial w}$$

$$= \frac{y^T y - y^T X\hat{w} - \hat{w}^T X^T y + \hat{w}^T X^T X \hat{w}}{\partial w}$$

$$= -X^T y - X^T y + 2X^T X \hat{w}$$

$$= 2X^T (X\hat{w} - y)$$

$$= 2X^T X \hat{w} - 2X^T y = 0$$

重要矩阵求导公式：

$$\frac{\partial AX}{\partial X} = A^T \qquad \frac{\partial AX^T}{\partial X} = A$$

$$\frac{\partial X^T A X}{\partial X} = (A^T + A)X$$

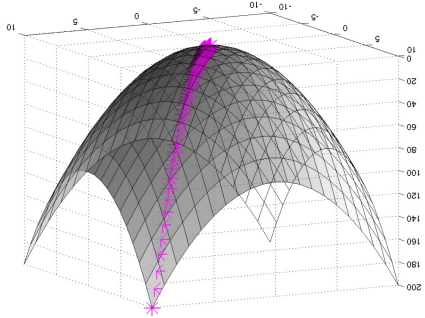
$$\hat{w} = (X^T X)^{-1} X^T y$$

因此：$f(\hat{x}_i) = \hat{w}^T \hat{x}_i = \hat{x}_i^T \hat{w} = \hat{x}_i^T (X^T X)^{-1} X^T y$

Learning as optimization

- The main idea
  - Make two assumptions:
    - the classifier is linear ✓
    - we want to find the classifier $\boldsymbol{\theta}$ that "fits the data best"
  - Formalize as an optimization problem
    - Pick a loss function $J(\boldsymbol{\theta}, \mathcal{D})$, and find $\text{argmin}_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
    - OR: Pick a "goodness" function, often $\Pr(\mathcal{D}|\boldsymbol{\theta})$, and find the $\text{argmax}_{\boldsymbol{\theta}} f_{\mathcal{D}}(\boldsymbol{\theta})$

Learning as optimization with gradient ascent

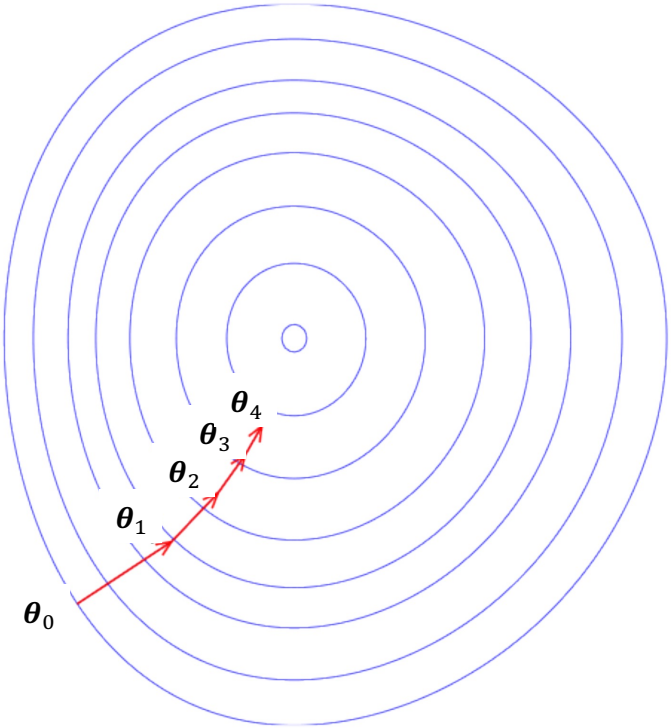


- **Goal:** Learn the parameter $\boldsymbol{\theta}$ of …
- **Dataset:** $\mathcal{D} = \{(\boldsymbol{x}_1, y_1) \ldots, (\boldsymbol{x}_m, y_m)\}$
- Use your model to define
  - $\Pr(\mathcal{D}|\boldsymbol{\theta}) = \ldots.$
- Set $\boldsymbol{\theta}$ to maximize Likelihood
  - Usually we use numeric methods to find the optimum
  - i.e., **gradient ascent:** repeatedly take a small step in the direction of the gradient (direction of fastest increase in the function).

Gradient ascent

梯度下降的过程

- To find $\text{argmax}_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$:
  - Start with $\boldsymbol{\theta}_0$
  - For $t = 1....$
    - $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha\, f'(\boldsymbol{\theta}_t)$

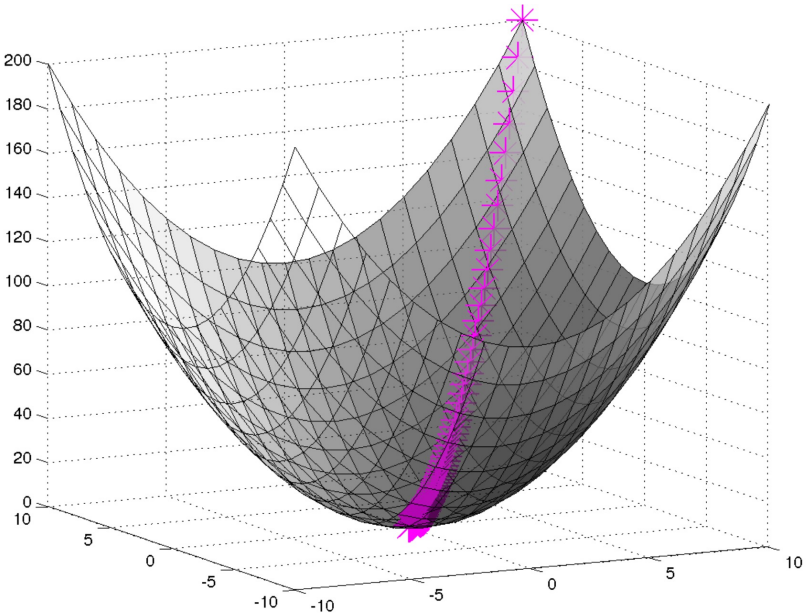


  - where α is a learning rate.
    - The larger it is, the faster $\boldsymbol{\theta}$ changes.
    - The values of α are typically small, e.g. 0.01 or 0.0001

Slide courtesy of William Cohen

Gradient descent

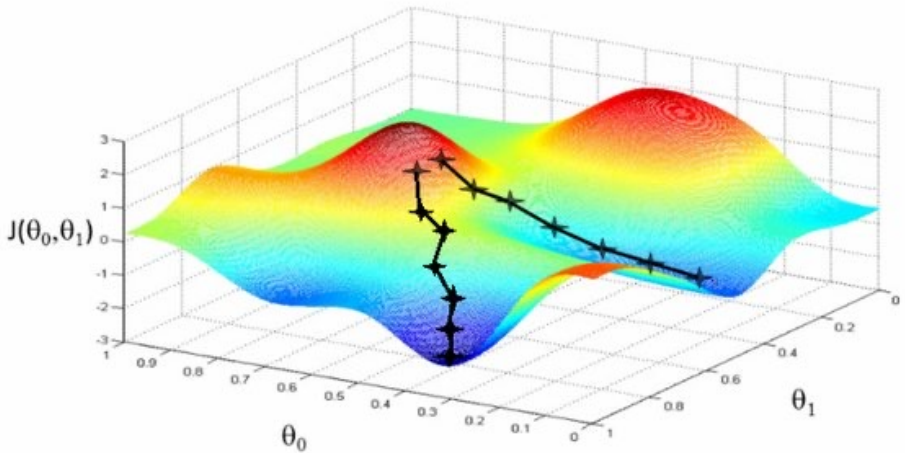
- Likelihood: ascent
- Loss: descent



Slide courtesy of William Cohen

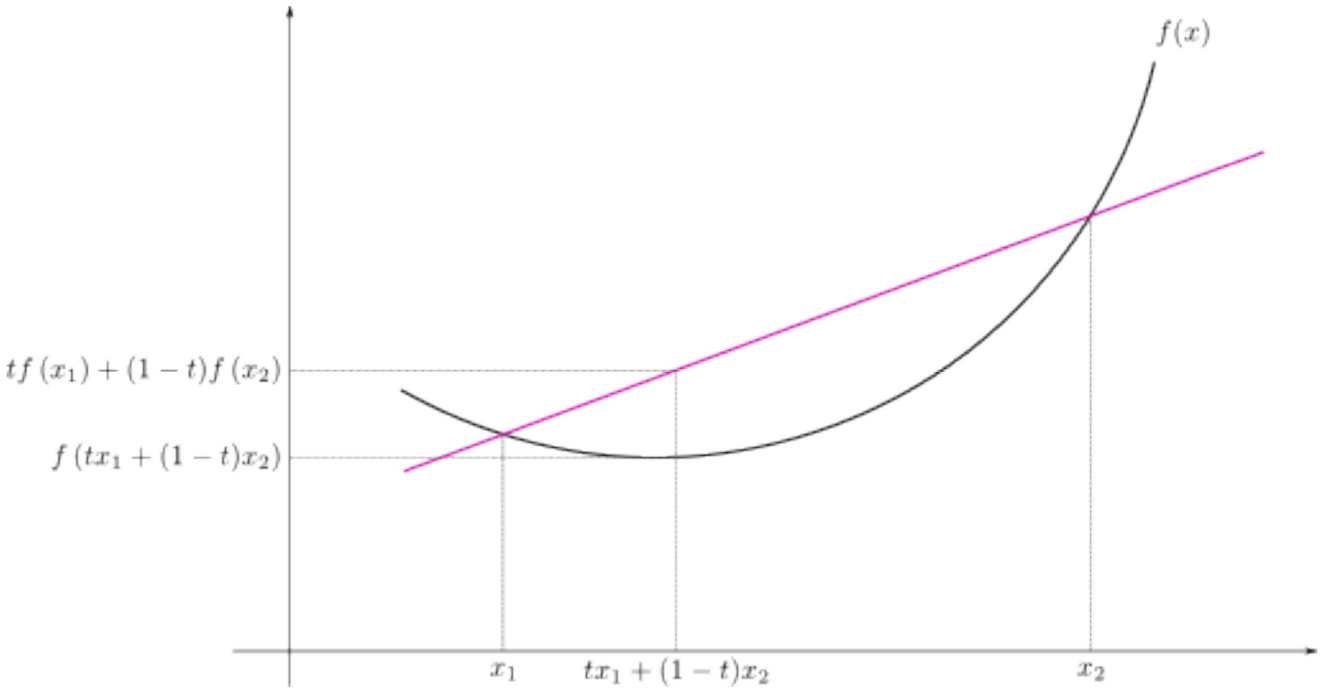
Pros and cons of gradient descent

- Simple and often quite effective
- Often very scalable
- Only applies to smooth functions (differentiable)
- Might find a local minimum, rather than a global one



Pros and cons of gradient descent

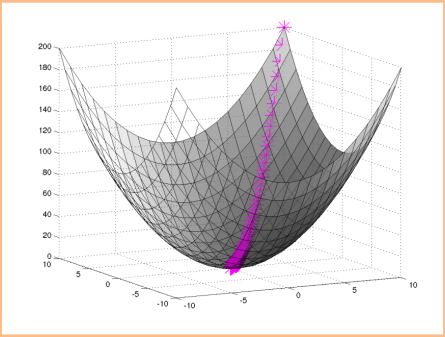
There is only one local optimum if the function is convex



Gradient Descent

Algorithm 1 Gradient Descent

1: **procedure** GD($\mathcal{D}$, $\boldsymbol{\theta}^{(0)}$)
2: $\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^{(0)}$
3: $\quad$**while** not converged **do**
4: $\quad\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
5: $\quad$**return** $\boldsymbol{\theta}$



In order to apply GD to Linear Regression all we need is the **gradient** of the objective function (i.e. vector of partial derivatives).

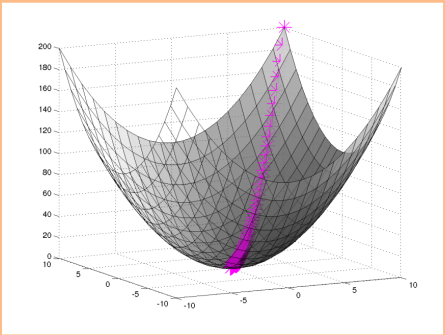
求导

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{d}{d\theta_1} J(\boldsymbol{\theta}) \\ \frac{d}{d\theta_2} J(\boldsymbol{\theta}) \\ \vdots \\ \frac{d}{d\theta_N} J(\boldsymbol{\theta}) \end{bmatrix}$$

Gradient Descent

Algorithm 1 Gradient Descent

1: **procedure** GD($\mathcal{D}$, $\boldsymbol{\theta}^{(0)}$)
2: $\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^{(0)}$
3: $\quad$**while** not converged **do**
4: $\quad\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
5: $\quad$**return** $\boldsymbol{\theta}$



There are many possible ways to detect **convergence**. For example, we could check whether the L2 norm of the gradient is below some small tolerance.

$$||\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})||_2 \leq \epsilon$$

停止条件

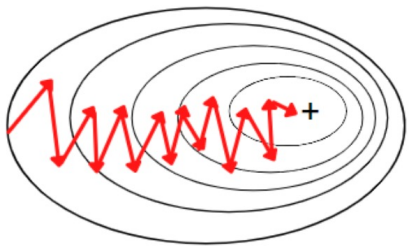
Alternatively we could check that the reduction in the objective function from one iteration to the next is small.

Stochastic Gradient Descent (SGD)

随机梯度，每次随机选择一小部分样本

Algorithm 2 Stochastic Gradient Descent (SGD)

1: **procedure** SGD($\mathcal{D}$, $\boldsymbol{\theta}^{(0)}$)
2: $\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^{(0)}$
3: $\quad$**while** not converged **do**
4: $\quad\quad$**for** $i \in$ shuffle($\{1, 2, \ldots, N\}$) **do**
5: $\quad\quad\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J^{(i)}(\boldsymbol{\theta})$
6: $\quad$**return** $\boldsymbol{\theta}$



Applied to Linear Regression, SGD is called the **Least Mean Squares (LMS)** algorithm

We need a per-example objective:

$$\text{Let } J(\boldsymbol{\theta}) = \sum_{i=1}^{m} J^{(i)}(\boldsymbol{\theta}) \qquad \text{where } J^{(i)}(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta}^T \boldsymbol{x}^{(i)} - y^{(i)})^2.$$

Partial Derivatives forLinear Reg.

Let $J(\boldsymbol{\theta}) = \sum_{i=1}^{m} J^{(i)}(\boldsymbol{\theta})$ where $J^{(i)}(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta}^T \boldsymbol{x}^{(i)} - y^{(i)})^2$.

$$
\begin{aligned}
\frac{d}{d\theta_k} J^{(i)}(\boldsymbol{\theta}) &= \frac{d}{d\theta_k} \frac{1}{2} (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2 \\
&= \frac{1}{2} \frac{d}{d\theta_k} (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2 \\
&= (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) \frac{d}{d\theta_k} (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) \\
&= (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) \frac{d}{d\theta_k} (\sum_{k=1}^{K} \theta_k x_k^{(i)} - y^{(i)}) \\
&= (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_k^{(i)}
\end{aligned}
$$

Partial Derivatives for Linear Reg.

GD 可以看成 SGD 的求和

Let $J(\boldsymbol{\theta}) = \sum_{i=1}^{m} J^{(i)}(\boldsymbol{\theta})$ where $J^{(i)}(\boldsymbol{\theta}) = \frac{1}{2}(\boldsymbol{\theta}^T \boldsymbol{x}^{(i)} - y^{(i)})^2$.

$$\frac{d}{d\theta_k} J^{(i)}(\boldsymbol{\theta}) = (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_k^{(i)}$$

Used by SGD (aka. LMS)

$$\frac{d}{d\theta_k} J(\boldsymbol{\theta}) = \frac{d}{d\theta_k} \sum_{i=1}^{N} J^{(i)}(\boldsymbol{\theta})$$

$$= \sum_{i=1}^{N} (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_k^{(i)}$$

Used by Gradient Descent

Least Squares

Learning: Three approaches to solving $\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- **Approach 1:** Closed Form

  (set derivatives equal to zero and solve for parameters)
  - pros: one shot algorithm!
  - cons: does not scale to large datasets (matrix inverse is bottleneck)
- **Approach 2:** Gradient Descent

  (take larger – more certain – steps opposite the gradient)
  - pros: conceptually simple, guaranteed convergence
  - cons: batch, often slow to converge
- **Approach 3:** Stochastic Gradient Descent (SGD)

  (take many small steps opposite the gradient)
  - pros: memory efficient, fast convergence, less prone to local optima
  - cons: convergence in practice requires tuning and fancier variants

Gradient Descent

- Why gradient descent, if we can find the optimum directly?
  - GD can be applied to a much broader set of models
  - GD can be easier to implement than direct solutions, especially with automatic differentiation software
  - For regression in high-dimensional spaces, GD is more efficient than
  - direct solution (matrix inversion is an $O(D^3)$ algorithm).

Probabilistic Interpretation of LMS

概率解释：最大似然推导出 LMS

- Let us assume that the target variable and the inputs are related by the equation:

$$y_i = \boldsymbol{\theta}^T \boldsymbol{x}_i + \epsilon_i$$

where ϵ is an error term of unmodeled effects or random noise

- Now assume that ϵ follows a Gaussian $\mathcal{N}(0, \sigma^2)$, then we have:

$$p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(y_i - \boldsymbol{\theta}^T\boldsymbol{x}_i)^2}{2\sigma^2}\right)$$

- By independence assumption:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{m} p(y_i|\boldsymbol{x}_i;\boldsymbol{\theta}) = \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^m \exp\left(-\frac{\sum_{i=1}^{m}(y_i - \boldsymbol{\theta}^T\boldsymbol{x}_i)^2}{2\sigma^2}\right)$$

Probabilistic Interpretation of LMS

- Hence the log-likelihood is:

$$l(\boldsymbol{\theta}) = m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{i=1}^{m} (y_i - \boldsymbol{\theta}^T \boldsymbol{x}_i)^2$$

- Do you recognize the last term?

Yes it is: $$J(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{m} (y_i - \boldsymbol{\theta}^T \boldsymbol{x}_i)^2$$

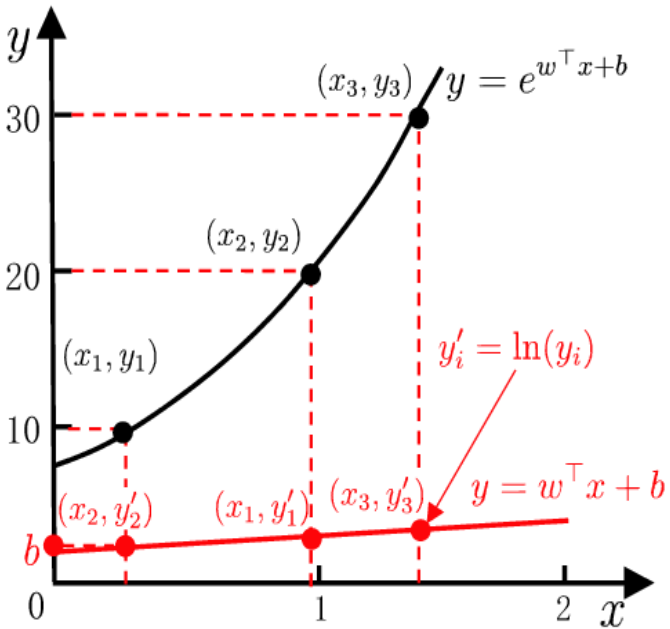
- Thus under independence assumption, LMS is equivalent to MLE of $\boldsymbol{\theta}$!

Although the model assumes a Gaussian distribution in the prediction (i.e. Gaussian noise function or error function), there is no such expectation for the inputs to the model ($\boldsymbol{x}$).

Log-Linear Regression

非线性变化系构建线性模型

- The logarithm of the output label can be used for approximation



$$\ln y = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$$

$$\Updownarrow$$

$$y = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$$